

CS G526: Advanced Algorithms and Complexity

Lecture 14

Sequence Alignment in Linear Space via Divide and Conquer

Instructor: Prof. Aniket Basu Roy

Scribes: Kunta Vikranth Reddy and Mayank Singh Bisht

11th September 2026

1 Sequence Alignment in Linear Space

In previous lectures, we learned how to compute the optimal alignment between two strings X and Y of lengths m and n , respectively. Building up the 2-D $m \times n$ array and formulating the dynamic programming algorithm, the running time is found to be $O(mn)$, and the space requirement is also $O(mn)$, since it was dominated by the cost of storing the array. In this lecture, we will see that the space requirement can be improved to $O(m + n)$, and how this affects our ability to recover the actual alignment.

$O(mn)$ Space Drawback. While an $O(mn)$ running time is often acceptable in practice, the $O(mn)$ space requirement can become a serious bottleneck as the input size grows. This is because the algorithm requires storing the entire two-dimensional table in memory simultaneously. For example, aligning two strings of length 100,000 would require storing on the order of 10 billion table entries costing several gigabytes of memory. This makes space the primary limiting factor for large-scale sequence alignment problems.

This approach via Divide and Conquer provides an enhancement of the sequence alignment algorithm that makes it work in $O(mn)$ time using only $O(m + n)$ space. i.e. linear space while still retaining the optimal alignment.

1.1 Space Efficient Algorithm

If we only care about the value of the optimal alignment, and not the alignment itself, it is easy to get away with linear space. We can observe that to fill in an entry of an array A , we only need information from the current column and the previous column of the A .

Algorithm: Space-Efficient-Alignment

Input: Strings X (length m), Y (length n); gap penalty δ ; mismatch cost α
Output: Array B containing the optimal alignment cost

```

1 Array  $B[0 \dots m, 0 \dots 1]$ 
2 Initialize  $B[i, 0] = i\delta$  for each  $i$                                 // just as in column 0 of  $A$ 
3 for  $j = 1, \dots, n$  do
4    $B[0, 1] = j\delta$                                                 // since this corresponds to entry  $A[0, j]$ 
5   for  $i = 1, \dots, m$  do
6      $B[i, 1] = \min [\alpha_{x_i y_j} + B[i - 1, 0], \delta + B[i - 1, 1], \delta + B[i, 0]]$ 
7   Move column 1 of  $B$  to column 0 to make room for next iteration:
8   Update  $B[i, 0] = B[i, 1]$  for each  $i$ 
```

Working. The Space-Efficient-Alignment algorithm computes the same optimal alignment cost as the standard dynamic programming algorithm, but instead of maintaining the entire $(m + 1) \times (n + 1)$ table A in memory, it maintains only two columns at a time, reflected in the array declaration $B[0 \dots m, 0 \dots 1]$.

Column 0 of B is initialized as $B[i, 0] = i\delta$ for each i , matching column 0 of the full table A and representing the cost of aligning the first i characters of X with an empty string. The algorithm then processes the columns of Y one at a time via an outer loop over $j = 1, \dots, n$, where B holds only the column currently being computed (index 1) and the column from the previous iteration (index 0).

For each j , the boundary entry $B[0, 1] = j\delta$ is set first, corresponding to $A[0, j]$, before the rest of the column is filled using an inner loop over $i = 1, \dots, m$ with the recurrence

$$B[i, 1] = \min [\alpha_{x_i y_j} + B[i - 1, 0], \delta + B[i - 1, 1], \delta + B[i, 0]].$$

The three terms correspond, respectively, to a diagonal move (aligning x_i with y_j), a vertical move (aligning x_i with a gap), and a horizontal move (aligning a gap with y_j) - exactly the standard recurrence $A[i, j] = \min[\alpha + A[i - 1, j - 1], \delta + A[i - 1, j], \delta + A[i, j - 1]]$, expressed in terms of the two retained columns. Once column j is fully computed, the algorithm overwrites index 0 with the values from index 1, i.e., $B[i, 0] = B[i, 1]$ for each i , so it becomes the previous column for the next iteration.

Problem with Space Efficient Alignment Algorithm. Since B at the end of the algorithm only contains the last two columns of the original dynamic programming array A , if we were to try tracing back to get the path, we would run out of information after just these two columns.

1.2 Divide and Conquer approach

The Divide and Conquer approach uses Space-Efficient-Alignment and Backward-Space-Efficient-Alignment together to find the optimal alignment.

Backward Space Efficient Alignment :

We know that $f(i, j)$ denotes the minimum cost of a path from $(0, 0)$ to (i, j) from the earlier Dynamic Programming approach i.e.

$$f(i, j) = \min(\alpha(x_i, y_j) + f(i-1, j-1), \delta + f(i-1, j), \delta + f(i, j-1))$$

Now, let's define $g(i, j)$ to be the minimum cost of the path from (i, j) to (m, n) . The function g provides an equally natural dynamic programming approach to sequence alignment, except that we build it up in reverse. $g(i, j)$ can be defined as :

$$g(i, j) = \min(\alpha(x_{i+1}, y_{j+1}) + g(i+1, j+1), \delta + g(i, j+1), \delta + g(i+1, j))$$

Similar to how $f(i, j)$ was used in the dynamic programming approach earlier to solve for sequence alignment, $g(i, j)$ can also be used in a reverse manner to solve for sequence alignment.

And in the same way as mentioned above, there is a space-efficient version of this backward dynamic programming algorithm, analogous to Space-Efficient-Alignment, which computes the value of the optimal alignment using only $O(m+n)$ space.

We will call this backwards space-efficient version as **Backward-Space-Efficient-Alignment**.

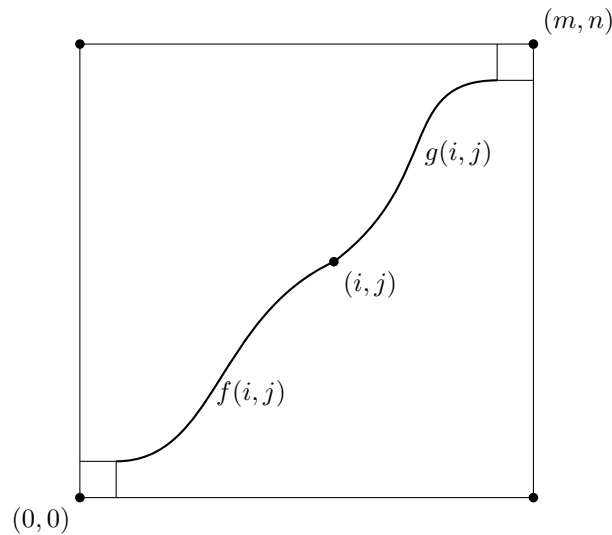


Figure 1: Dynamic programming path decomposition.

Lemma 1. *The length of the shortest path from $(0,0)$ to (m,n) that passes through (i,j) is $f(i,j) + g(i,j)$.*

Proof. Let L^* be the length of the shortest path.

Claim 1: $L^* \leq f(i,j) + g(i,j)$ [True by definition of L^*]

Claim 2: $L^* \geq f(i,j) + g(i,j)$

For the sake of contradiction, if $L^* < f(i,j) + g(i,j)$

Then either $f(i,j)$ is not the shortest path from $(0,0)$ to (i,j) or $g(i,j)$ is not the shortest path from (i,j) to (m,n) or both.

Hence, both claims are true via conjunction,

$$L^* = f(i,j) + g(i,j)$$

□

Lemma 2.

$$\forall j^* \in \{0, \dots, n\}, \exists i^* : i^* = \arg \min_{i \in \{0, \dots, m\}} \{f(i, j^*) + g(i, j^*)\}$$

$\mathcal{E}$ the shortest path from $(0,0)$ to (m,n) passes through (i^, j^*) .*

Proof. Let L^* be the length of the shortest path from $(0,0)$ to (m,n)

Claim 1: $L^* \leq f(i^*, j^*) + g(i^*, j^*)$ [True by definition of L^*]

Claim 2: $L^* \geq f(i^*, j^*) + g(i^*, j^*)$

For the sake of contradiction, if $L^* < f(i^*, j^*) + g(i^*, j^*)$

Then either $f(i^*, j^*)$ is not the shortest path from $(0,0)$ to (i^*, j^*) or $g(i^*, j^*)$ is not the shortest path from (i^*, j^*) to (m,n) or both.

Hence, both claims are true via conjunction,

L^* is the shortest path from $(0,0)$ to (m,n)

□

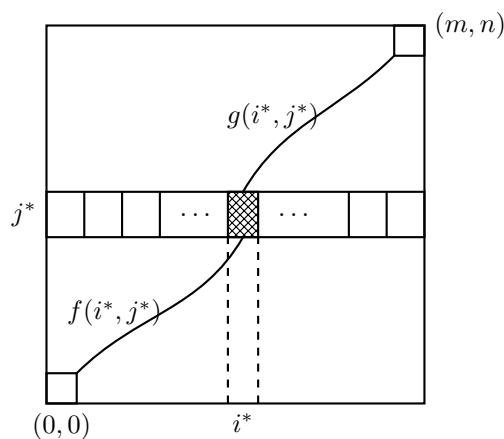


Figure 2: Dynamic programming path decomposition-2

Implementation. Using lemma 2 and our space-efficient algorithms to compute the value of the optimal alignment, we will proceed as follows.

We divide the DP table along its center column and compute the value of $f(i, n/2)$ and $g(i, n/2)$ for each value of i , using our two space-efficient algorithms.

We can then determine the minimum value of $f(i, n/2) + g(i, n/2)$, and conclude via Lemma 2 that there is a shortest corner-to-corner path passing through the node $(i, n/2)$. Given this, we can search for the shortest path recursively in the portion between $(0, 0)$ and $(i, n/2)$ and in the portion between $(i, n/2)$ and (m, n) .

The crucial point is that we apply these recursive calls sequentially and reuse the working space from one call to the next. Thus, since we only work on one recursive call at a time, the total space usage is $\mathbf{O(m + n)}$.

In running the algorithm, we maintain a globally accessible list P which will hold nodes on the shortest corner-to-corner path as they are discovered.

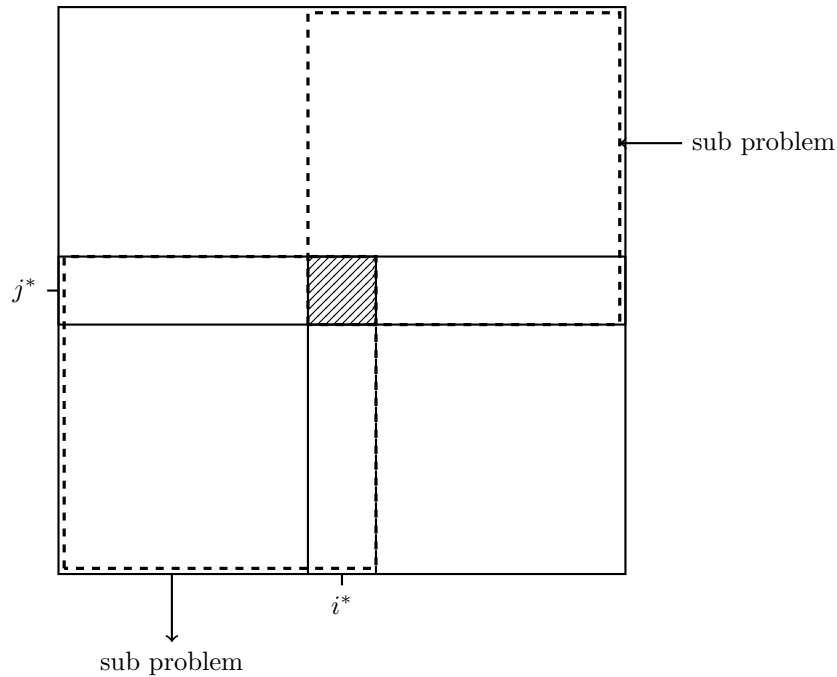


Figure 3: The first level of recurrence for the space-efficient Divide-and-Conquer-Alignment. The two outlined regions indicate the input to the two recursive cells.

Algorithm: Divide-and-Conquer-Alignment(X, Y)

Input: Sequence X of length m , Sequence Y of length n

Output: Global alignment path P

- 1 **if** $m \leq 2$ **or** $n \leq 2$ **then**
 - 2 $\perp$ Compute optimal alignment using Alignment(X, Y)
 - 3 Call Space-Efficient-Alignment($X, Y[1 : n/2]$)
 - 4 Call Backward-Space-Efficient-Alignment($X, Y[n/2 + 1 : n]$)
 - 5 Let q be the index minimizing $f(q, n/2) + g(q, n/2)$
 - 6 Add $(q, n/2)$ to global list P
 - 7 Divide-and-Conquer-Alignment($X[1 : q], Y[1 : n/2]$)
 - 8 Divide-and-Conquer-Alignment($X[q + 1 : n], Y[n/2 + 1 : n]$)
 - 9 **return** P
-

1.3 Time Complexity

Let $T(m, n)$ denote the maximum running time of the algorithm on strings of length m and n . The algorithm performs $O(mn)$ work to build up the arrays B and $\bar{B}$, and then recurses on strings of size q and $n/2$, and on

strings of size $m - q$ and $n/2$. Thus, for some constant c and some choice of index q ,

$$\begin{aligned} T(m, n) &\leq cmn + T(q, n/2) + T(m - q, n/2), \\ T(m, 2) &\leq cm, \\ T(2, n) &\leq cn \end{aligned}$$

To guess the form of the solution, we considered the restrictive special case $m = n$ with a perfect bisection $q = n/2$. Writing T as a function of the single variable n gives

$$T(n) \leq 2T(n/2) + cn^2$$

which yields

$$T(n) = O(n^2)$$

Motivated by this, we conjecture that in general $T(m, n)$ grows like the product mn , and guess $T(m, n) \leq kmn$ for some constant k , to be proved by induction.

The base cases $m \leq 2$ and $n \leq 2$ hold as long as $k \geq c/2$. Assuming $T(m', n') \leq km'n'$ for all pairs with a smaller product, we obtain

$$T(m, n) \leq cmn + T(q, n/2) + T(m - q, n/2) \leq cmn + \frac{kqn}{2} + \frac{k(m - q)n}{2} = cmn + \frac{kmn}{2} = \left(c + \frac{k}{2}\right) mn.$$

Choosing $k = 2c$ makes the inductive step go through, completing the proof that

$$\boxed{T(m, n) = O(mn)}$$